

×

 Introduction
Video • 10 min

✔ Flipped Interaction Pattern
Video • 7 min

 The Agent Loop
Video • 8 min

Knowledge Check: Agent Loop & Flipped Interaction
Practice Assignment • Grade: 100%

Running the Code Samples in the Course
Reading • 10 min

 **Programmatic Prompting for Agents**
Ungraded Plugin • 15 min

- Try Out Programmatic Prompting
Reading • 10 min

Programmatic Prompting for Agents II
Ungraded Plugin • 15 min

Programmatic Prompting for Agents III
Ungraded Plugin • 15 min

Programmatic Prompting: Building Blocks for AI Agents
Dialogue • 15 min

Try Out the Customer Service Agent
Reading • 10 min

● Giving Agents Memory
Ungraded Plugin • 15 min

Practicing Programmatic Prompting for Agents
Ungraded Plugin • 15 min

Practicing Programmatic Prompting for Agents
(Solution)
Ungraded Plugin • 15 min

Adding Structure to AI Agent Outputs
Video • 14 min

- Building Your First Agent
Ungraded Plugin • 15 min

AI Agent / Environment Interface
Ungraded Plugin • 15 min

AI Agent Feedback and Memory
Ungraded Plugin • 15 min

The Agent Loop: From Understanding to Thinking Like an Agent
Dialogue • 15 min

Learning More & Staying Connected
Reading • 10 min

Understanding Agentic AI Concepts
Graded Assignment • 30 min

[Expand](#)

✓ Completed

Go to next item

Sending Prompts Programmatically & Managing Memory

To get started building agents, we need to understand how to send prompts to LLMs. Agents require two key capabilities:

1. Programmatic prompting - Automating the prompt-response cycle that humans do manually in a conversation. This forms the foundation of the Agent Loop we'll explore.
2. Memory management - Controlling what information persists between iterations, like API calls and their results, to maintain context through the agent's decision-making process.

Programmatically sending prompts is how we move from having a human type in prompts and then take action based on the LLM's response to having an agent that can do this automatically. The Agent Loop that we will begin building over the next several readings will be programmatically sending prompts to the LLM and then taking action based on the LLM's response.

We will also need to understand how to manage what the LLM knows or remembers. This is important because we want to be able to control what information the LLM has in each iteration of the loop. For example, if it just called an API, we want it to remember what API it asked to be invoked and what the result of that action was.

Basic Usage

Here's a simple example of how to send prompts to an LLM using LiteLLM:

```
#
# Set your API key as shown in the Google Colab notebook
#

from litellm import completion
from typing import List, Dict

def generate_response(messages: List[Dict]) -> str:
    """Call LLM to get response"""
    response = completion(
        model="openai/gpt-4o",
        messages=messages,
        max_tokens=1024
    )
    return response.choices[0].message.content

messages = [
    {"role": "system", "content": "You are an expert software engineer that prefers functional programming."},
    {"role": "user", "content": "Write a function to swap the keys and values in a dictionary."}
]

response = generate_response(messages)
print(response)
```

Now, go to the next item, and try out the code in the Google Colab notebook.